Gabarito P2

1) hipótese 1: Se ~~estou~~ determinar o menor elemento no vetor de tamanho (n-1)

algoritmo menor1 (A[n])

```
    se n==1 então
        retorne (1)
    senão
        m = menor (A[n-1])
        se A[m] < A[n] então
            retorne (m)
        senão
            retorne (n)
        fim se
    fim se
fim algoritmo
```

2) O maior elemento do subvetor A[1..j-1] está na posição j-1. (menor)

3) Invariante do laço: "O elemento k não está no subvetor A[1..i-1].
Inicialização: i vale 1 logo o subvetor A[1..i-1] é o subvetor vazio, obviamente o invariante é verdadeiro antes da primeira iteração.

Manutenção: em cada iteração o algoritmo verifica se i indica um elemento no vetor, (i≤n), uma vez que i indica ~~pro~~ primeiro elemento fora do subvetor A[1..i-1], e também verifica se o elemento A[i] é k,

$A[i] \neq k$, sendo $A[i]$ diferente de k, o elemento i é "descartado" no subvetor que não contém k, quando o algoritmo incrementa i na linha 3. e assim o invariante se mantém verdadeiro. Mas, no caso do elemento i ser k i não é incrementado mantendo o invariante verdadeiro.

Finalização: O laço encerra por um de dois motivos, ou porque i é maior que n ou porque $A[i]$ é k. No primeiro caso i é igual a $n+1$ e o invariante é verdadeiro uma vez que $A[1..i-1]$ é $A[1..n]$ e k não está no vetor. No segundo caso, i indica a posição do elemento k, e o invariante $A[1..i-1]$ não contém o elemento k que está no subvetor $A[1..i]$, logo o invariante ~~está~~ se mantém verdadeiro e o algoritmo está correto.

4)

```
algoritmo buscabinR (A, i, f, k)
    se i ≠ f então
        m = (f+i)/2
        se A[m] ≠ k então
            se A[m] > k então
                retorne (buscabinR (A, 1, m-1, k)
            senão
                retorne (buscabinR (A, m+1, f)
            fim se
        senão
            retorne (m)
        fim se
```

2 2

$\cancel{T(n) = T(n-1) + n}$

$\cancel{T(n-1) + n \leq cn}$

$\cancel{c(n-1) + n \leq cn}$

$cn - c + n \leq cn$ falso.

$\cancel{n(c+1) - c \leq cn}$

② ①

```
senão
    se A[i] = x então
        retorne (i)
    senão
        retorne (i-1)
    fim se
fim se
fim algoritmo
```